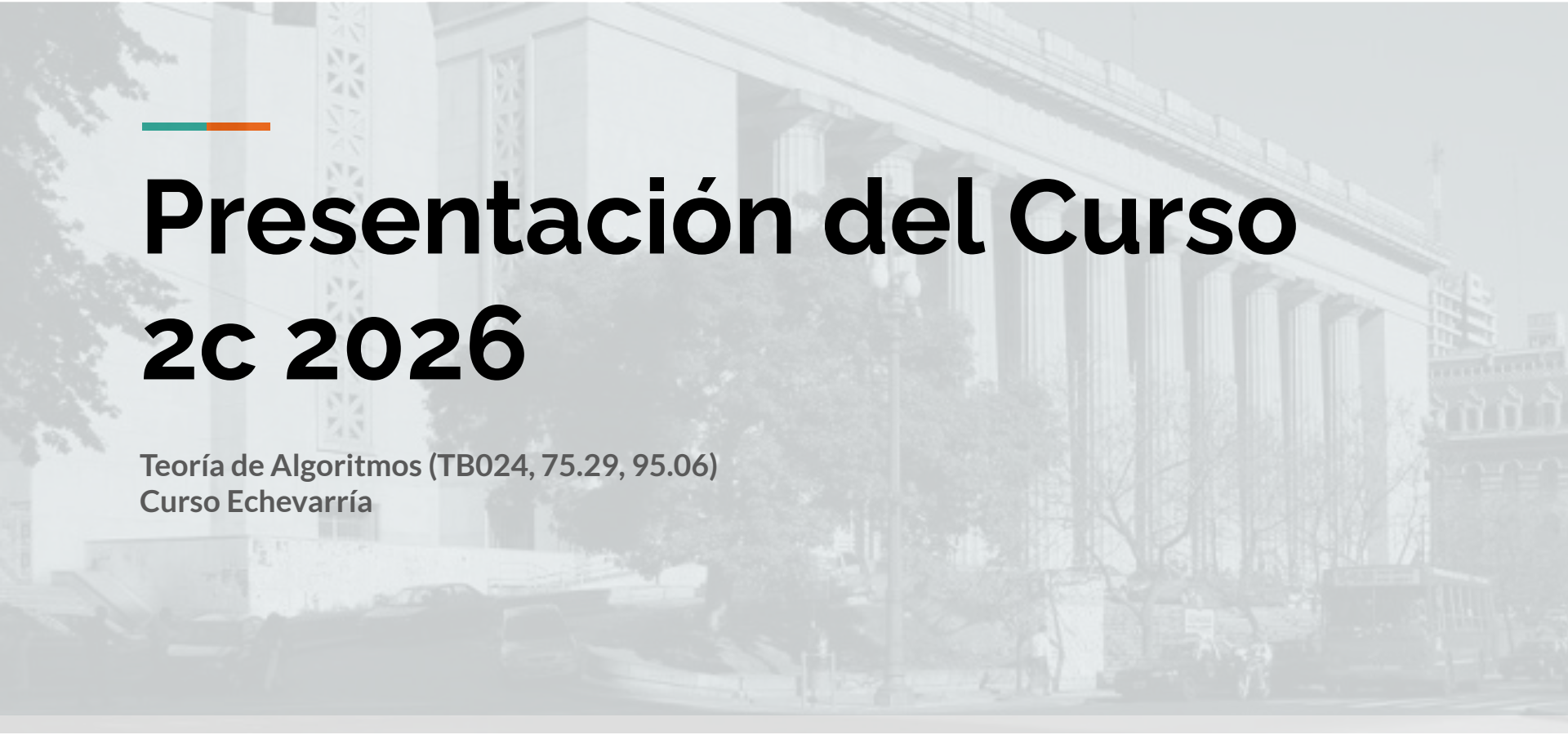


Presentación del Curso 2c 2026

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Docentes

Profesor: Pablo Echevarría

JTP: Germán Marín

Ayudantes:

- Rubén Bohórquez
- Daniela Bolívar
- Leandro Brizuela
- Rocío Bustamante
- Juan Cardoso
- Felipe D'Alto
- Nicolás Mazzeo
- Juan Ignacio Moore
- José Patty Morales

Modalidad de las Clases

- Clases Teórico-Prácticas
- Martes
  - 19 a 22 hs
  - Presencial (Aula 506)
- Jueves
  - 19 a 22 hs
  - Virtual Sincrónico (Meet)
  - Sugerencia: ingresar con el mail @fiuba
- En general, las clases se graban y se publican un par de días después



Arrancamos Puntualmente

Régimen de Cursada

- Asistencia:
  - Es obligatoria: aunque no tomamos lista, traten de venir
- Aprobación de la Materia
  - Aprobar el TP Individual
  - Aprobar los 2 TP Grupales
  - Aprobar el Parcial
  - Aprobar autoevaluaciones
- Parcial
  - 1 Parcial Escrito
  - 2 Instancias de Recuperación
- TP Individual
  - 1 TP0: Individual
- TP Grupales
  - 2 Trabajos Prácticos Grupales
    - TP1: 1 Reentrega
    - TP2: Defensa oral
- Autoevaluación
  - Lenguajes y autómatas

Materias Correlativas

- TBA024 (Ingeniería en Informática, Plan 2020, actualización 2023)
  - Algoritmos y Estructuras de Datos
  - Introducción al Desarrollo de Software
- 75.29 (Ingeniería en Informática, Plan 1986, actualización 2011)
  - 75.41 Algoritmos y Programación II
- 95.06 (Licenciatura en Análisis de Sistemas, Plan 1986, actualización 2014)
  - 81.11 Matemática Discreta
  - 95.15 Algoritmos y Programación II



Completar Historial Académico en el Campus Virtual

Calificación y Nota Final

- Calificación de cada Entrega del TP1 y TP2
- Calificación del Parcial
- Nota Final
  - 50% Parcial + (50% Promedio simple de TPs Grupales +/- Incidencia TP Individual)

Canales de Comunicación

- Clases
- Slack
- Campus Virtual

Campus Virtual

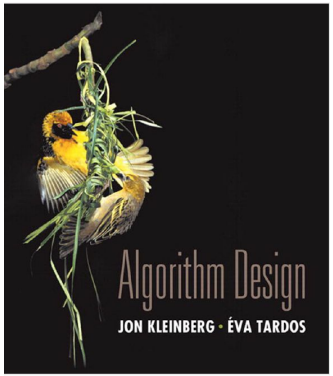
- Ya todos deberían estar matriculados
- Bibliografía
- Calendario
- Enunciado y Entregas del TP

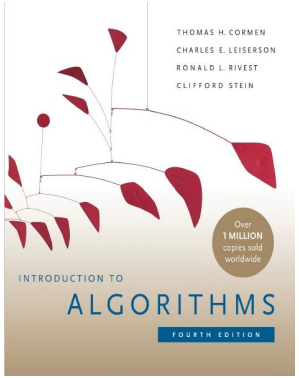
Cronograma Teoría de Algoritmos – 2do Cuatrimestre 2026

Sem	Fecha (martes)	Martes (Presencial)	Bibliografía	Fecha (jueves)	Jueves (Virtual Sincrónico)	Bibliografía	Hitos
1	18/8/2026	Presentación de la materia. Introducción. Fuerza Bruta. Notación. Repaso.	KT[2,3]; CL[10-12,20,22,B4]; SI[0]	20/8/26	División y Conquista	KT[5]; CL[4]	Enunciado TP0 (Mar 18/8)
2	25/8/2026	Algoritmos Greedy I	KT[4]; CL[15]	27/8/26	Algoritmos Greedy II	ER[2]	Entrega TP0 (Sab 29/8)
3	1/9/2026	Devolución TP0. Backtracking I		3/9/26	Backtracking II	ER[2]	Devol. TP0 (Ma 1/9) Enunciado TP1 (Jue 3/9)
4	8/9/2026	**Práctica D&C, Greedy y Backtracking.**		10/9/26	Programación Dinámica I	KT[6]; CL[14]; TA[12]	
5	15/9/2026	Programación Dinámica II	KT[6]; CL[14]; TA[12]	17/9/26	Programación Dinámica III	KT[6]; CL[14]; TA[12]	
6	22/9/2026	**Práctica Programación Dinámica.**		24/9/26	Programación Lineal I	CL[29]; TA[2,9]	
7	29/9/2026	Programación Lineal II	CL[29]; TA[2,9]	1/10/26	Redes de Flujo. Ford-Fulkerson	KT[7]; CL[24]	Entrega TP1 (Dom 4/10)
8	6/10/2026	Redes de Flujo. Aplicaciones	KT[7]; CL[24]	8/10/26	Complejidad y Reducciones. P vs NP.	KT[8-9]; CL[34]; S[7-8]	
9	13/10/2026	**Práctica PL y Redes de Flujo.**		15/10/26	Complejidad. NP-Completo.	KT[8-9]; CL[34]; S[7-8]	Devolución TP1 (Dom 18/10)
10	20/10/2026	**Práctica Reducciones. Repaso para el parcial**		22/10/26	Algoritmos de Aproximación I	KT[11]; CL[35]	
11	27/10/2026	**[Parcial]**		29/10/26	Algoritmos de Aproximación II	KT[11]; CL[35]	
12	3/11/2026	Algoritmos de Aproximación III –		5/11/26	Algoritmos Randomizados I	KT[13]; CL[5]; KA	Enunciado TP2 (Mar 3/11)
13	10/11/2026	Algoritmos Randomizados II	KT[13]; CL[5]; KA	12/11/26	**Práctica Alg. de Aproxi-mación y Randomizados**		Notas parcial (Mar 10/11)
14	17/11/2026	**[1er Recuperatorio]**		19/11/26	Autómatas, Lenguajes y Gramáticas	SI[1-2]	
15	24/11/2026	Computabilidad y Máquinas de Turing	SI[3-5]	26/11/26	**Práctica Autómatas y Computabilidad**		Entrega TP2 (Dom 29/11)
16	1/12/2026	**Defensa TP2**		3/12/26	**Defensa TP2 (PRESENCIAL)**		Notas Recu (Mar 1/12) Cierre autoev (Vie 4/12)
		FERIADO		10/12/26	**[2do Recuperatorio]** (Notas el 15/12)		

Material para las Clases

- Calendario
- Bibliografía en el Calendario y en el Campus Virtual
- Guía de TP: *work in progress*





Links Útiles

- Aula Virtual (Campus): https://campusgrado.fi.uba.ar/course/view.php?id=1330
- Drive: https://drive.google.com/drive/u/3/folders/1KarLqknqmyIMZvTDdXGdrgmBJyvnaGUL
- Videos de las Clases: https://www.youtube.com/watch?v=F28EyEumjfk&list=PLyj8Dq9az_0NKoemTT74rjzFWKCMmW6iW&pp=gAQB

Recomendaciones

- Practicar

El que suda más
en la práctica,
sangra menos
en la guerra

Credo del soldado espartano



Recomendaciones

- Practicar, practicar



“Oigo y olvido,
veo y recuerdo,
hago y comprendo”
Confucio

Recomendaciones

- Practicar, practicar, practicar



Teoría de Algoritmos



Teoría de Algoritmos en resumen

- Vamos a ver algoritmos y estrategias para resolver algunos tipos de problemas.
- Problemas complejos: Se sabe cómo resolverlos pero se tarda mucho.
- Problemas combinatorios: La cantidad de posibles soluciones es inmanejable.
- Complejidades relativas entre problemas.

Primer problema: Send more money

- Un estudiante norteamericano está estudiando en otra ciudad y se le está acabando la plata. Le envía un telegrama a su padre que dice “SEND MORE MONEY” (Envía más dinero)
- El padre le responde que le va a enviar más dinero si le demuestra que está aprovechando las clases de TDA y crea un algoritmo que encuentre los valores que resuelvan la siguiente ecuación:

```
  S E N D
+
  M O R E
----------
M O N E Y
```

Primer problema: Send more money

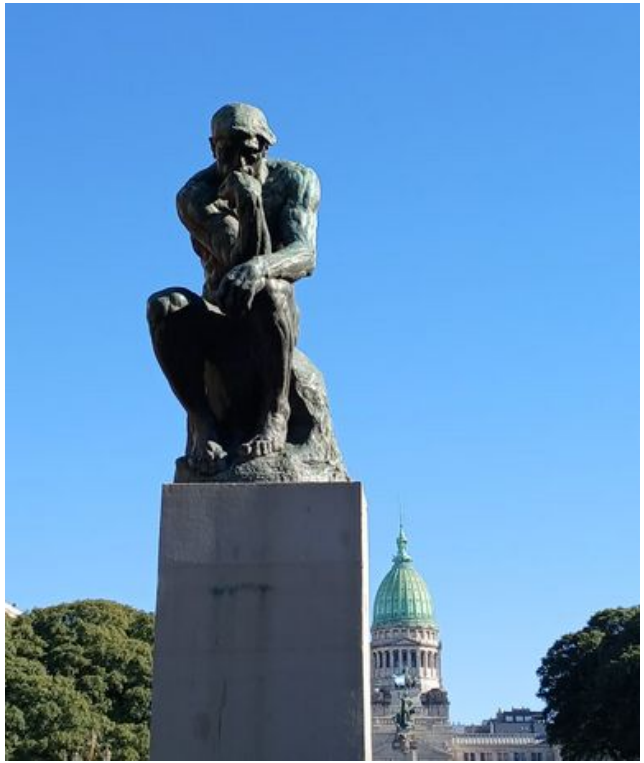
SendMoreMoneyFB.py

```
import time

def money():
    t1=time.time()
    for s in range (0,10):
        for e in range (0,10):
            for n in range (0,10):
                for d in range (0,10):
                    for m in range (0,10):
                        for o in range (0,10):
                            for r in range (0,10):
                                for y in range (0,10):
                                    send = s*1000+e*100+n*10+d
                                    more = m*1000+o*100+r*10+e
                                    money = m*10000+o*1000+n*100+e*10+y
                                    if (y!=s and y!=e and y!=n and y!=d and y!=m and y!=o and y!=r and r!=o and r!=m and
                                        r!=d and r!=n and r!=e and r!=s and o!=m and o!=d and o!=n and o!=e and o!=s and
                                        m!=d and m!=n and m!=e and m!=s and d!=n and d!=e and d!=s and n!=e and n!=s and
                                        e!=s and m!=0 and s!=0):
                                        if send + more == money:
                                            print("SEND =", send)
                                            print("MORE =", more)
                                            print("MONEY =", money)
    t2=time.time()
    print(t2-t1)
money()
```

Send more money: Complejidad

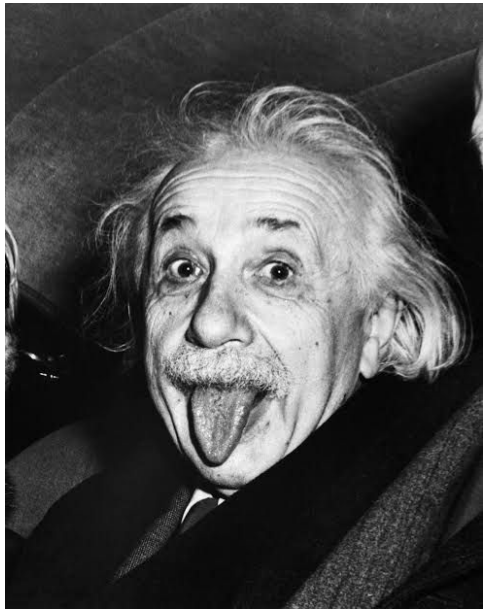
- Este código tarda 75 seg en hallar la respuesta.
- ¿Eso es bueno o es malo?



“El Pensador” - A. Rodin

Complejidad

- El tiempo es relativo
- Nos interesa la cantidad de instrucciones que tengamos que ejecutar.
- Son 8 fors que se ejecutan 10 veces cada uno.
- ¿Cuantas veces se ejecutan las instrucciones de adentro?
- 10x10x10….. → 10^8
- Para una clave de n letras, 10^n
- A medida que agrego más letras, el incremento del tiempo es exponencial.



Complejidad

- Los problemas se dividen en dos grandes grupos: polinomiales (“fáciles”) y exponenciales (“difíciles”)

$$\lim_{n \to \infty} \frac{n^b}{a^n} = 0$$

- Este límite se cumple para todo a>1 y para todo b
- O sea, a^n crece más rápido que n^b
- O sea, cualquier exponencial crece más rápido que cualquier polinomial.

- $\log_k(n) < n$ para todo n>1 → polinomial
- $n! < n^n$ → exponencial

Complejidad: Notación

- Un algoritmo requiere 3 n2 + 2n + 5 pasos otro, requiere 2 n2 + 3000n + 200 pasos.
- Sólo nos interesa el orden de magnitud: n2

- Crecimiento asintótico: Notación
- **O(n2): Indica que la función crece a lo sumo al ritmo n2**
- **Ω**(n2): Indica que la función crece por lo menos al ritmo n2
- ⊖(n2): Indica que la función crece exactamente al ritmo n2

(Siempre para el peor caso)

Fuerza Bruta



Créditos: Cottonbro Studio

Fuerza Bruta

¿De qué se trata?

- Buscar la solución a un problema combinatorio explorando (y analizando) todas las soluciones existentes

Es la forma más simple de resolver un problema, pero también es la más ineficiente

Fuerza Bruta (cont.)

Un algoritmo genérico de fuerza bruta tiene más o menos esta forma:

1. Repetir:
    a. Generar la próxima solución
    b. Probar si la solución es válida
        i. Si es válida, ver si es mejor que la mejor solución hasta ahora.
2. Mientras queden soluciones sin probar.

- Funciona mientras el problema tenga una cantidad finita de soluciones, posibles de enumerar.
- Pero es terriblemente lento.
- Vamos a ver estrategias para evitar probar todas las soluciones.

Fuerza Bruta (cont.)

¿Cómo se resolvería por Fuerza bruta…

- … el Problema del Viajante?
  - plantearíamos todas las posibles combinaciones de ciclos
  - calcularíamos el peso para cada uno
  - seleccionaríamos el de menor peso
- … el Problema de la Mochila?
  - plantearíamos todas las posibles combinaciones
  - descartaríamos las que exceden el peso máximo
  - seleccionaríamos la que maximiza el valor total
- … el Problema de Interval Scheduling?
  - plantearíamos todas las combinaciones de tareas
  - descartaríamos las que tienen solapamiento
  - seleccionaríamos la que tenga mayor cantidad de tareas

Estrategias

- División y conquista
- Algoritmos Greedy
- Backtracking
- Programación dinámica
- Grafos y redes de flujo
- Programación lineal
- Reducciones
- Algoritmos randomizados
- Heurísticas
- Autómatas



Trabajo Práctico 0

Cuartetos de Números Primos

Los números 11,13, 17 y 19 tienen una característica interesante. Son 4 números primos y están en la misma decena. El siguiente cuarteto que cumple esa condición es (101, 103, 107 y 109).

Se realizó un programa que calcula y muestra todos los cuartetos de números primos menores que 1.000.000. Este programa es eficaz, pero no eficiente: tarda 31 horas en llegar a un millón.

Se pide diseñar e implementar mejoras al programa para que sea más eficiente. NO SE DEBE DESARROLLAR UN NUEVO PROGRAMA: sólo modificar el original.

Requisitos de la Entrega

1. Supuestos: identificar supuestos, limitaciones, condiciones o premisas bajo los cuales funcionará el algoritmo.
2. Diseño: Pseudocódigo y estructuras de datos utilizadas.
3. Análisis de complejidad: realizar un análisis de complejidad temporal del algoritmo existente y del modificado.
4. Tiempos de Ejecución: medir y graficar los tiempos de ejecución para los números de 1 a 200000, 1 a 400000, 1 a 600000, 1 a 800000 y 1 a 1000000 por lo menos. Se pueden agregar otros valores que se consideren relevantes.
5. Informe de Resultados: redactar un informe de resultados comparando los tiempos de ejecución. ¿Se corresponde con la complejidad temporal determinada inicialmente?

Trabajo Práctico 0

```python
import time
def esprimo(n):
    i = 2
    cd=0
    while i<=n-1:
        if n%i == 0:
            cd = cd+1
        i = i+1
    if cd ==0:
        return True
    else:
        return False
def prog():
    t1=time.time()
    for i in range (11,1000000):
        if esprimo(i) and esprimo(i+2) and esprimo(i+6) and esprimo(i+8):
            print (i, i+2, i+6,i+8)
    t2=time.time()
    print(t2-t1)

prog()
```

Trabajo Práctico 0

Condiciones Generales de Entrega

- El trabajo es individual. Animamos a que discutan alternativas con sus compañeros, pero el desarrollo e implementación de las mejoras y la entrega es individual.
- Fecha límite de entrega: sábado **29 de agosto a las 23:59** por campus
- Si no se entrega, se entrega fuera de plazo, o se entrega algo que no funciona, está desaprobado.
- **Debe correr en Python 3.10**
  - Entregar un único archivo .py con apellido y padrón en el nombre (TP0_Martínez_73289.py o algo así)
- **La reingeniería debe ser transparente: No cambiar el prototipo de la función**
- Los 10 mejores tiempos(*), tienen +1 en la nota final de los TPs (**).
- Los 5 peores tiempos(*), tienen -1 en la nota final de los TPs
- El trabajo debe ser entregado en un archivo zip conteniendo:
  - Documento con carátula, índice y numeración de páginas. La carátula debe incluir nombre y padrón. Debe presentarse en formato PDF.
  - Archivos con el código fuente del algoritmo desarrollado. Incluir instrucciones para compilar (de ser necesario) y ejecutar.
  - Archivo con los resultados obtenidos
- Aquí no toleramos el plagio: incluir referencias bibliográficas, utilizar normas APA 7ma edición

(*) Los tiempos obtenidos en mi computadora serán los únicos válidos para la evaluación

(**) No sirve para convertir un TP desaprobado en aprobado. La nota final de TP no puede ser mayor a 10



¡Muchas gracias!